declaration files reduces package size and improves compatibility, but means users cannot access or modify the original implementation, and requires keeping declarations in sync with the code. Including TypeScript source directly provides transparency and better tree-shaking, but increases package size and may require additional build tool configuration, potentially limiting compatibility for pure JavaScript users.

For popular JavaScript libraries without builtin types, DefinitelyTyped provides type definitions published under the `@types` scope, installable as dev dependencies. The TypeScript compiler also automatically looks for type definitions in the `node_modules/@types` folder. For legacy code or custom global variables, custom `.d.ts` files can be created and placed in directories such as `src/types` or `types`. Adding type support for a third-party JavaScript library involves identifying the library, analyzing its structure and exports, and creating a `.d.ts` file with accurate type definitions using `declare module`. Once defined, test the file in TypeScript code to ensure proper type checking and tooling support, and optionally contribute to DefinitelyTyped for widely used libraries.

TypeScript's module and global declaration files differ in scope and purpose: module declaration files provide type information for specific libraries or modules, making types available only on import, while global declaration files define types accessible project-wide. Using module declarations for modular code and global declarations for project-wide needs keeps the codebase organized and maintainable.

In essence, `.d.ts` files serve as the bridge that enables TypeScript to safely understand and interact with the vast JavaScript ecosystem, overlaying a typed layer on untyped code. Their use fosters a culture of code quality and consistency, as all team members rely on shared type contracts—making collaboration smoother and refactoring safer and more predictable. Furthermore, automated documentation tools can leverage these declaration files, ensuring documentation stays in sync with the codebase and reducing manual maintenance efforts.

For this topic, are you able to answer these follow-up/probing questions?

- "How do declaration files enable TypeScript to provide static type